
accelerate
collaborate
innovate
communicate
palm
PalmSource
Conference 2000



Full-Motion Animation

David Oakley,CTO
Howard Tomlinson,CEO
Astraware Ltd.

PalmSource
Conference 2000

Introduction

- Background
  - What is animation?
  - Special considerations on Palm OS® handhelds
- Graphics Techniques
  - What does Palm OS provide?
  - Business applications vs. full-screen animation
- Animation on Palm OS® Handhelds
  - Creating smooth, flicker-free animation
  - Getting more (and more) speed

What Is Animation?

- Animated cartoon (Merriam-Webster, 1915)
  - "A motion picture made from a series of drawings simulating motion by means of slight progressive changes in the drawings"
- Speed
  - About 30 frames (drawings) per second (FPS) is required to fool the brain that motion is continuous
- Progression
  - The changes have to be big enough to see, without being so large that the jumps are unrealistic



Limitations of Palm OS® Devices

- Operating system support
  - Very simple graphic primitives only
  - No circles, triangles, polygons
  - No 3D
  - Compatibility over speed
- Device limitations
  - Small, low-resolution screen
  - Persistent screen on greyscale devices (a plus?)
  - Slow processor (relatively)

Graphics in a Standard Business Application

- Business applications don’t need fast graphics
- Screen is composed of a window plus elements such as buttons and input fields
  - Handled automatically by the OS
- Changes to the screen image are generally in response to user interaction
  - A delay of less than 0.1s perceived by the user as instantaneous

What Is a Sprite and How Are They Created?

- A sprite is a discrete graphical image that can move on the screen
- A simple, rectangular sprite can be drawn using **`WinCopyRectangle`**…
- …But usually the sprite has an irregular outline requiring a ‘mask’ which allows the background to show through



- Sprites often have a ‘z-index’, determining which is in front when several sprites overlap
  - The Painter’s Algorithm

Animation in Your Application Code

- Initialisation code
  - Load the graphics you'll need in your animation
  - Set up the positions of your sprites
  - Store the value of **`TimGetTicks`**, which will be used to periodically 'fire' new frames
- In event loop
  - Use **`EvtGetEvent`** with a timeout of 0 or 1 ticks
  - When any event is fired (inc. **`nilEvent`**s), check whether it is time to start a new frame
  - Refresh screen if a new frame is required

Be Prepared (to Go Faster)

- Business applications tend to use **`WinDrawBitmap`** throughout
  - This is a slow function, esp. on compressed bitmaps
  - Very little flexibility
- Programs using animation should only use this to load off-screen windows on start-up (using **`WinCreateOffscreenWindow`**)
- Once the off-screen window has been loaded, application can then use **`WinCopyRectangle`**
  - Fast
  - Many drawing options, for masks, etc.

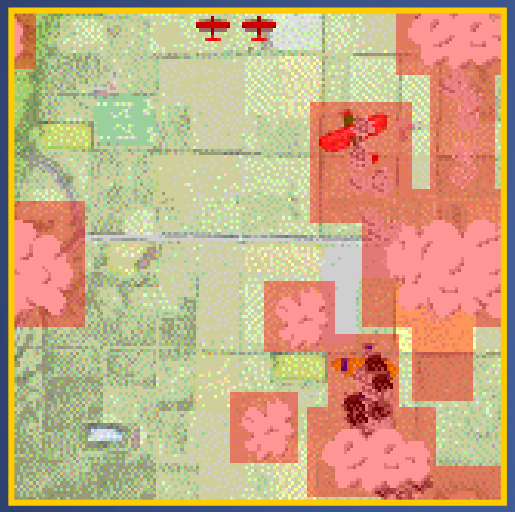
Building Up a Frame

- Easiest way—redraw the screen completely
  - Wipe it blank
  - Draw each sprite from furthest to nearest
  - Easy—but lots of redundant drawing if not all the sprites move



Building Up a Frame

- Harder way—only redraw the changes
  - Work out which areas have changed
  - Update each section that has changed
  - Potentially faster, as less graphics drawn, but a real headache to work out



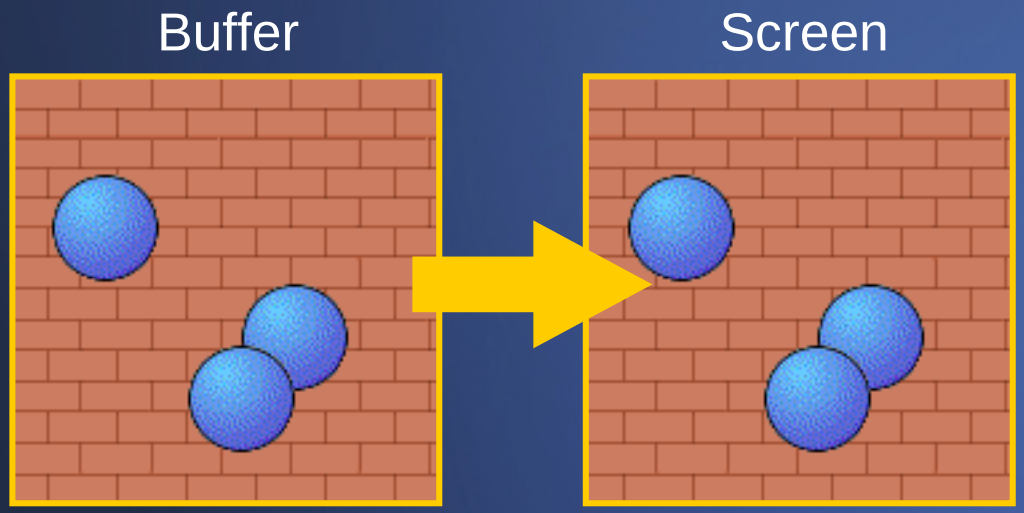
- Trade-off—which is appropriate…

Using Buffering to Reduce Flicker

- If you write straight to the screen…
  - The user sees the screen get ‘blanked’ each time
  - The user sees the graphics added one by one
  - The final graphic is only on screen for a short time before the next refresh (and so flickers the most)

Using Buffering to Reduce Flicker

- If you write to a buffer first…
  - User doesn't see the graphics being built
  - The previous image stays onscreen, and you replace once the new image has been completed
  - The user only sees the small differences between each frame, and is more likely to accept it as a smooth animation



PalmSource Conference 2000

Animating the Palm OS® Legal Way

- Use **`WinCopyRectangle`** over and over…
  - To draw background and images to buffer
  - To copy buffer to screen
  - And it even knows how to do it for different depths!
- You *should* get sufficient performance to convince people
  - Providing you aren't trying to move too many graphics around per frame
- Benefits of stopping here
  - Palm, Inc. approval
  - Device independence—only one build required

PalmSource Conference 2000

Breaking the Law

- Palm OS® was written for device independence
  - Written in C for generic hardware…
  - …For the lowest common denominator
- You know your App better than Palm, Inc. does!
  - Special case—copying the whole screen
  - Cut corners that Palm, Inc. wouldn't
- Access the screen directly
  - It is just a chunk of memory (at the moment)
  - Use MemMove or your own C or 68000
- Detect hardware/OS, and exit gracefully

Other Optimization Options

- Write your own ‘blitting’ code
  - Specifically for 1-,2-,4-,8-,16-bit depth sprite drawing
  - Writing direct to off-screen buffers is legal
- Understand the bottlenecks
  - Where does your App spend most of its time?
  - Apps often spend 90% of their time in 10% of the code—optimize that 10% first
- Pre-generate as much as possible
  - Such as sprites with several layers
  - But bear in mind memory restrictions

Summary

- Prepare your graphics on start-up
- Optimize for speed
- Stick to the Palm OS® routines if at all possible

- More resources at: http://www.astraware.com/palmsource2000/

Thank You!



www.astraware.com

Q&A


accelerate
collaborate
innovate
communicate
palm
PalmSource
Conference 2000